Experiment No. 3

Aim: Implementation of a given problem statement/s for different types of inheritance.

1. **Write a Java program to create a class known as "BankAccount" with methods called deposit() and withdraw(). Create a subclass called SavingsAccount that overrides the withdraw() method to prevent withdrawals if the account balance falls below one hundred.**

```
class BankAccount {
    protected double balance;
    public BankAccount(double balance) {
        this.balance = balance;
    }
    public void deposit(double amount) {
        if (amount > 0) {
            balance += amount;
            System.out.println("Deposited: " + amount);
            System.out.println("Balance: " + balance);
        } else {
            System.out.println("Invalid deposit amount.");
        }
    }
    public void withdraw(double amount) {
        if (amount > balance) {
            System.out.println("Insufficient balance.");
        } else {
            balance -= amount;
            System.out.println("Withdrawn: " + amount);
            System.out.println("Balance: " + balance);
```

```
        }
    }
}
class SavingsAccount extends BankAccount {
    public SavingsAccount(double balance) {
        super(balance);
    }
    @Override
    public void withdraw(double amount) {
        if ((balance - amount) < 100) {
            System.out.println("Withdrawal denied! Minimum balance of 100 must be
maintained.");
        } else {
            balance -= amount;
            System.out.println("Withdrawn: " + amount);
            System.out.println("Balance: " + balance);
        }
    }
}
public class Main {
    public static void main(String[] args) {
        SavingsAccount sa = new SavingsAccount(500);
        sa.deposit(200);
        sa.withdraw(550);
        sa.withdraw(100);
    }
}
```

```
PS D:\Prasanna\exp3> javac Main.java
PS D:\Prasanna\exp3> java Main
Deposited: 200.0
Balance: 700.0
Withdrawn: 550.0
Balance: 150.0
Withdrawal denied! Minimum balance of 100 must be maintained.
PS D:\Prasanna\exp3>
```

2. **Write a Java program that creates a class hierarchy for employees of a company. The base class should be Employee, with subclasses Manager, Developer, and Programmer. Each subclass should have properties such as name, address, salary, and job title. Implement methods for calculating bonuses, generating performance reports, and managing projects**

```
class Employee {
    protected String name;
    protected String address;
    protected double salary;
    protected String jobTitle;
    public Employee(String name, String address, double salary, String jobTitle) {
        this.name = name;
        this.address = address;
        this.salary = salary;
        this.jobTitle = jobTitle;
    }
    public double calculateBonus() {
        return salary * 0.05;
    }
    public void generatePerformanceReport() {
        System.out.println("Performance report for " + name + " (" + jobTitle + ")");
    }
    public void manageProjects() {
        System.out.println(name + " is managing general tasks.");
    }
}
```

```
class Manager extends Employee {
    public Manager(String name, String address, double salary) {
        super(name, address, salary, "Manager");
    }
    @Override
    public double calculateBonus() {
        return salary * 0.20;
    }
    @Override
    public void generatePerformanceReport() {
        System.out.println("Manager " + name + " performance: Excellent leadership.");
    }
    @Override
    public void manageProjects() {
        System.out.println("Manager " + name + " is overseeing multiple projects.");
    }
}
class Developer extends Employee {
    public Developer(String name, String address, double salary) {
        super(name, address, salary, "Developer");
    }
    @Override
    public double calculateBonus() {
        return salary * 0.10;
    }
    @Override
    public void generatePerformanceReport() {
```

```
        System.out.println("Developer " + name + " performance: Strong coding skills.");
    }
    @Override
    public void manageProjects() {
        System.out.println("Developer " + name + " is working on development tasks.");
    }
}
class Programmer extends Employee {
    public Programmer(String name, String address, double salary) {
        super(name, address, salary, "Programmer");
    }
    @Override
    public double calculateBonus() {
        return salary * 0.08;
    }
    @Override
    public void generatePerformanceReport() {
        System.out.println("Programmer " + name + " performance: Efficient programming.");
    }
    @Override
    public void manageProjects() {
        System.out.println("Programmer " + name + " is handling coding assignments.");
    }
}
public class Main {
    public static void main(String[] args) {
        Manager m = new Manager("Amit", "Pune", 80000);
        Developer d = new Developer("Neha", "Mumbai", 60000);
```

```
        Programmer p = new Programmer("Ravi", "Nagpur", 50000);
        System.out.println("Manager Bonus: " + m.calculateBonus());
        m.generatePerformanceReport();
        m.manageProjects();
        System.out.println();
        System.out.println("Developer Bonus: " + d.calculateBonus());
        d.generatePerformanceReport();
        d.manageProjects();
        System.out.println();
        System.out.println("Programmer Bonus: " + p.calculateBonus());
        p.generatePerformanceReport();
        p.manageProjects();
    }
}
```

```
PS D:\Prasanna\exp3> javac Mainn.java
PS D:\Prasanna\exp3> java Mainn
Manager Bonus: 16000.0
Manager Amit performance: Excellent leadership.
Manager Amit is overseeing multiple projects.

Developer Bonus: 6000.0
Developer Neha performance: Strong coding skills.
Developer Neha is working on development tasks.

Programmer Bonus: 4000.0
Programmer Ravi performance: Efficient programming.
Programmer Ravi is handling coding assignments.
PS D:\Prasanna\exp3>
```

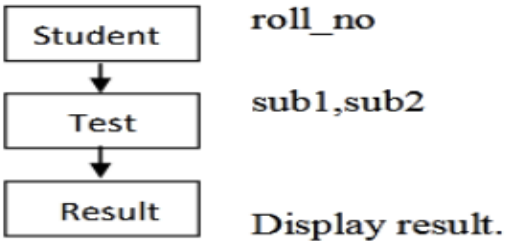
3. **Implement Following: a. Create abstract class shapes with dim1, dim2 variables and abstract area() method. Class b. rectangle and triangle inherits shape class. Calculate area of rectangle and triangle.**

```
abstract class Shape {
   protected double dim1;
   protected double dim2;
   public Shape(double dim1, double dim2) {
      this.dim1 = dim1;
      this.dim2 = dim2;
   }
   abstract double area();
}
class Rectangle extends Shape {
   public Rectangle(double dim1, double dim2) {
      super(dim1, dim2);
   }
   @Override
   double area() {
      return dim1 * dim2;
   }
}
class Triangle extends Shape {
   public Triangle(double dim1, double dim2) {
      super(dim1, dim2);
   }
   @Override
```

```
    double area() {
        return 0.5 * dim1 * dim2;
    }
}
public class Main {
    public static void main(String[] args) {
        Rectangle r = new Rectangle(10, 5);
        Triangle t = new Triangle(10, 5);
        System.out.println("Rectangle area: " + r.area());
        System.out.println("Triangle area: " + t.area());
    }
}
```

```
PS D:\Prasanna\exp3> javac Mainnn.java
PS D:\Prasanna\exp3> java Mainnn
Rectangle area: 50.0
Triangle area: 25.0
PS D:\Prasanna\exp3>
```

4. Write a program to perform Multilevel Inheritance




```
class Student {
    protected int roll_no;
    public void setRollNo(int roll_no) {
        this.roll_no = roll_no;
```

```
    }

    public void displayRollNo() {
        System.out.println("Roll No: " + roll_no);
    }
}
class Test extends Student {
    protected int sub1, sub2;
    public void setMarks(int sub1, int sub2) {
        this.sub1 = sub1;
        this.sub2 = sub2;
    }
    public void displayMarks() {
        System.out.println("Subject 1: " + sub1);
        System.out.println("Subject 2: " + sub2);
    }
}
class Result extends Test {
    public void displayResult() {
        int total = sub1 + sub2;
        double avg = total / 2.0;
        displayRollNo();
        displayMarks();
        System.out.println("Total: " + total);
        System.out.println("Average: " + avg);
    }
}
public class Main {
```

```
    public static void main(String[] args) {
        Result r = new Result();
        r.setRollNo(101);
        r.setMarks(85, 90);
        r.displayResult();
    }
}
```

```
PS D:\Prasanna\exp3> javac Mainnnn.java
PS D:\Prasanna\exp3> java Mainnnn
Roll No: 101
Subject 1: 85
Subject 2: 90
Total: 175
Average: 87.5
PS D:\Prasanna\exp3>
```